

Chapter 6

Partitioning

Splitting data across machines — and keeping load fair when one key owns the world

Source: [claude-skills / ch06-partitioning.md](#) · Illustrated · Interview-ready

Martin Kleppmann · Part II: Distributed Data

Table of Contents

1. Core Idea
2. Framework: Partitioning by Key Range
3. Framework: Partitioning by Hash of Key
4. Skew, Hot Spots & Key Salting
5. Compound Primary Key
6. Secondary Indexes: Local vs Global
7. Rebalancing Strategies
8. Request Routing & Coordination
9. Mental Models
10. Anti-Patterns
11. Worked Example: Sensor Network
12. Problems Each Mechanism Solves
13. Connects To (Cross-Chapter)
14. Key Takeaways
15. Junior SWE Checkpoints

1. Core Idea

Replication (Ch 5) **copies** data; partitioning **splits** it. When a dataset or query load is too big for one machine, break it into **partitions** (a.k.a. shards in MongoDB/Elasticsearch, regions in HBase, tablets in BigTable, vnodes in Cassandra/Riak) so each record belongs to exactly one partition, and spread partitions across nodes in a shared-nothing cluster.



Scale data volume

No single disk holds the whole dataset.



Scale query throughput

Parallelize reads/writes across nodes.



Fair load

Each node does roughly equal work — when partitioning works.

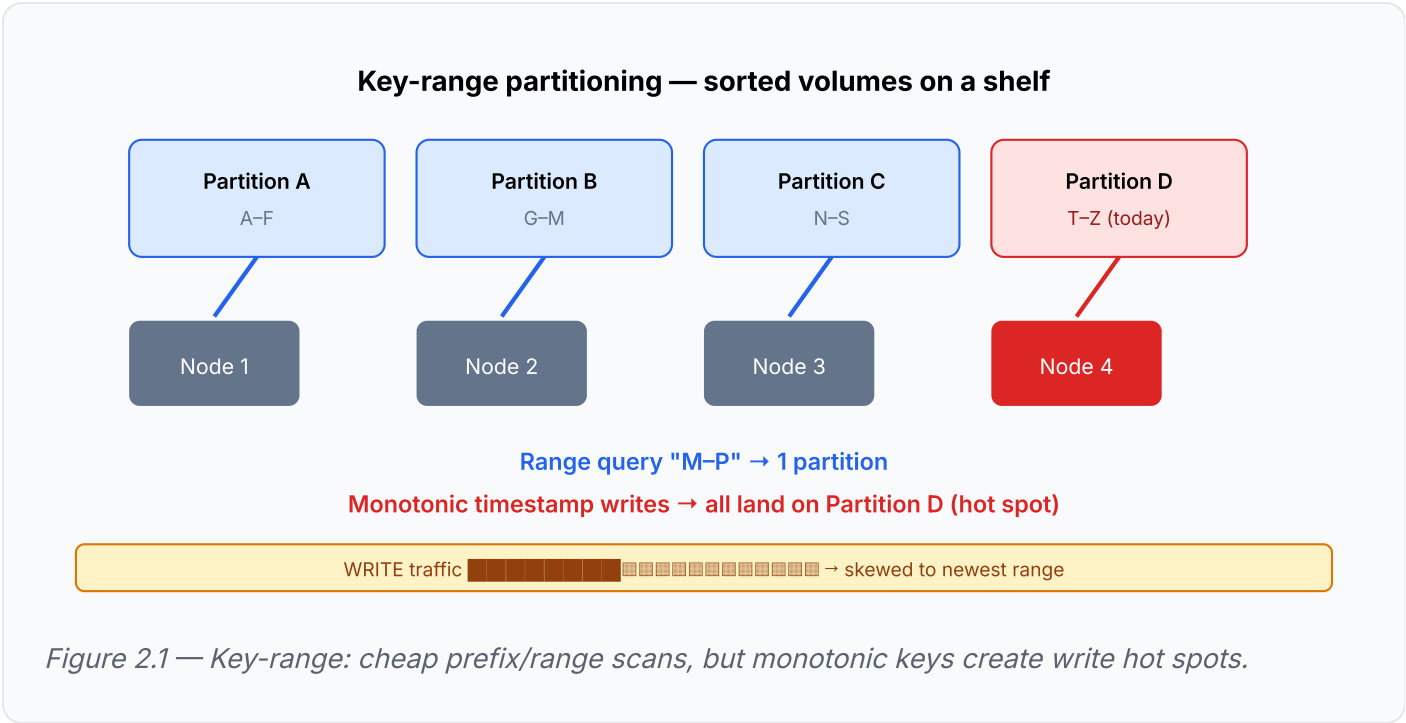
The whole game is spreading data and load evenly. Unfair partitioning is called **skew**; a partition with disproportionately high load is a **hot spot** — in the worst case one node does all the work while nine sit idle.

Partitioning + replication: usually combined — each node leads some partitions, follows others. But the two choices are *mostly independent*. Interview move: "Each shard has its own leader + followers" (Ch 5).

2. Framework: Partitioning by Key Range

Also: range sharding, encyclopedia volumes. **BigTable**, **HBase**, **RethinkDB**, early **MongoDB**.

When	You need efficient range scans (time-series reads, prefix lookups) and can tolerate hot-spot risk.
How	Assign each partition a contiguous range of sorted keys. Boundaries adapt to data density, chosen manually or automatically. Keys stay sorted within a partition (SSTable-style), so range queries hit one or few partitions.
Why it works	Sort order is preserved — the key doubles as a concatenated index.
Failure mode	Access pattern concentrates on one end of the range: a timestamp key means all of today's writes land on today's partition while the rest idle.



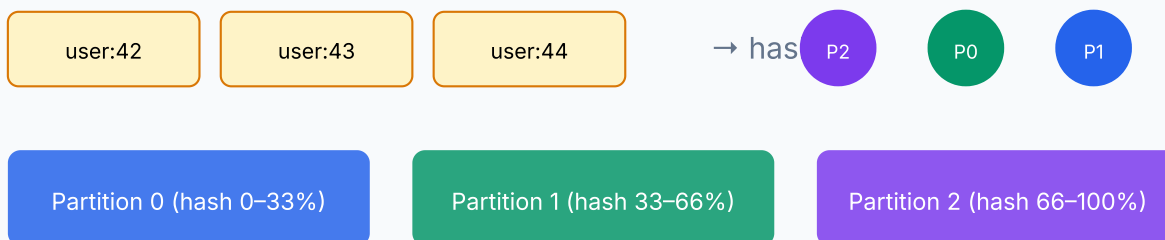
 **Encyclopedia shelf:** volumes sorted A–Z — easy to find a chapter by letter. But if everyone only reads the newest volume, that shelf sags while the rest collect dust.

3. Framework: Partitioning by Hash of Key

Default for write-heavy key-value workloads. MongoDB (hash mode), Riak, Couchbase, Voldemort, Cassandra.

When	Even distribution matters more than range queries.
How	Hash each key (MD5/FNV — doesn't need to be cryptographic), assign each partition a range of <i>hashes</i> . Similar keys get scattered uniformly.
Why it works	A good hash turns skewed inputs into a uniform distribution.
Failure mode	(a) Range queries — adjacent keys scattered, MongoDB hash mode sends range queries to <i>all</i> partitions; Riak/Couchbase/Voldemort don't support them on primary key. (b) Single hot key — identical keys → identical hashes → one partition hammered.

Hash partitioning — scatter keys, even load



Writes spread evenly across partitions

Range query "user:40-user:50" → must query ALL partitions (scatter/gather)

Figure 3.1 — Hash scatters adjacent keys; even write load, no cheap range scans on primary key.

Consistent hashing caveat: Karger et al.'s consistent hashing (pseudo-random boundaries, built for CDN caches) rebalances poorly for databases — Cassandra abandoned it pre-1.2. "Consistent" here has *nothing* to do with replica or ACID consistency. Kleppmann's advice: avoid the term; say **hash partitioning**.

4. Skew, Hot Spots & Key Salting

Skew = uneven load defeats partitioning. Hashing reduces but can't eliminate it — for a single hot key the application must intervene.

Problem	Example	Why hashing fails
Monotonic keys + range partitioning	Timestamp as sole key	All writes hit newest partition
Single celebrity / viral key	One user ID gets all traffic	hash(same key) = same partition every time

Application-level fix: key salting

Append a 2-digit random suffix to spread writes over 100 keys (e.g. `celebrity_id#42`). **Cost:** reads must gather all 100 variants; you must track which keys are split.

 **Hot key on hash partition:** A celebrity's ID still hammers one partition. No partition scheme fixes a single logical key — only the application can split it (salting) or cache aggressively at the edge.

5. Compound Primary Key

Cassandra-style: hash only the **first column** to pick the partition; remaining columns are a concatenated sort index within it.

```
(user_id, update_timestamp)
```

```
↑ hash this → pick partition
```

```
↑ sort within partition → cheap per-user time-range scans
```

Even distribution

Hash `user_id` → writes spread across users.

Efficient range scans

Within one partition, rows sorted by `timestamp`.

Scoped trade-off

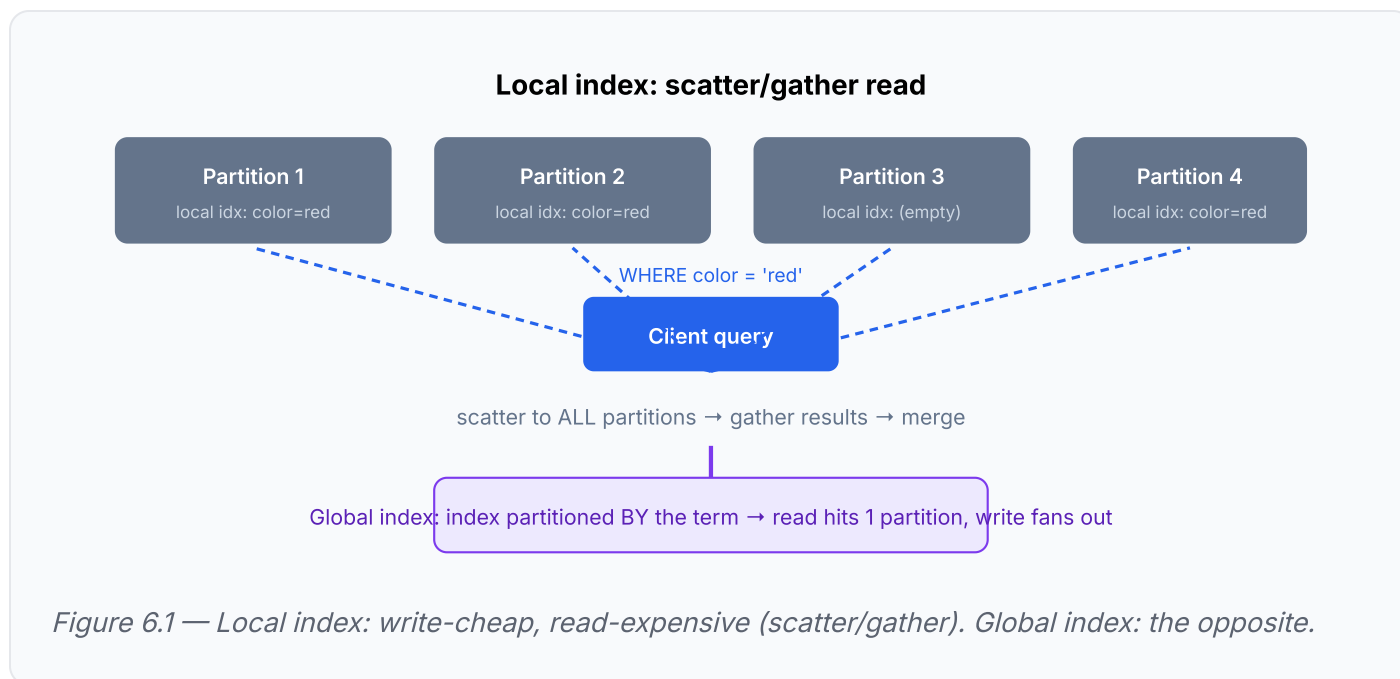
Multi-user time-range query = N targeted scans
(one per user), not one global scan.

Interview move: split "which partition" from "sort within partition." Hash the first component, sort by the rest. Used in chat systems `(channel_id, message_id)`, email `(user_id, ...)`.

6. Secondary Indexes: Local vs Global

Primary key picks the partition. Secondary indexes let you query by non-primary fields — but partitioning forces a painful trade.

Type	Also called	Writes	Reads	Systems
Document-partitioned	Local secondary index	Fast — touch one partition only	Scatter/gather across all partitions — tail latency amplification	MongoDB, Riak, Cassandra, Elasticsearch, SolrCloud, VoltDB
Term-partitioned	Global secondary index	Single doc update may touch many index partitions → typically async (DynamoDB GSIs)	Fast — read hits one partition	DynamoDB GSIs



Pick your pain: write-cheap / read-expensive (local index) *or* read-cheap / write-expensive (global index). Secondary indexes force this trade — there is no free option. Synchronous global index updates need distributed transactions (Ch 7 & 9) — which is why DynamoDB GSIs are async.

7. Rebalancing Strategies

Moving data between nodes as load or cluster size changes. Requirements: fair load after, keep serving reads/writes during, move no more data than necessary.

 **hash mod N (anti-pattern):** `hash(key) mod N` reassigns most keys whenever N changes — rebalancing becomes catastrophically expensive. **Never use this.**

Strategy	How	Best for	Caveat
Fixed number of partitions	Create far more partitions than nodes (e.g. 1,000 for 10 nodes); new nodes steal whole partitions. Key→partition mapping never changes, only partition→node.	Hash partitioning	Riak, Cassandra ≥1.2, Elasticsearch, Couchbase
Dynamic partitioning	Split when partition exceeds size threshold (HBase default 10 GB), merge when it shrinks — B-tree-like. Count adapts to data volume.	Key-range partitioning	Empty DB starts with one partition (one busy node) — mitigate with pre-splitting

Fixed partitions: move mapping, not key→partition

Before (3 nodes, 9 partitions)

P0 P1 P2

P3 P4 P5

P6 P7 P8

After (+1 node, steal partitions)

P0 P1

P2 P3

P4-P6

P7 P8

Key → partition mapping unchanged. Only partition → node moves.

hash(key) mod N → nearly ALL keys move when N changes ✗

Fixed partitions / dynamic splits → move only what's necessary ✓

Figure 7.1 — Good rebalancing changes partition→node assignment; hash mod N entangles key mapping with node count.

DDIA Visual Guide · ch06 partitioning · Page 9 of 19

Fully automatic rebalancing + automatic failure detection: a slow node gets declared dead, rebalancing dumps its load on the rest, overloading them → **cascading failure**. Couchbase/Riak/Voldemort keep a human commit step for this reason.

8. Request Routing & Coordination

Service discovery: how does a client find the right node for a key?

- 1 **Any-node routing** — client hits any node, which forwards to the partition owner
- 2 **Routing tier** — partition-aware proxy (mongos, moxi) sits between clients and storage nodes
- 3 **Partition-aware clients** — client library knows the partition map directly

The hard part: keeping everyone's view of **partition** → **node** assignment in agreement.

Coordination approach	Systems	Trade-off
ZooKeeper (or Helix)	HBase, SolrCloud, Kafka; Espresso via Helix	External dependency; authoritative partition map
Gossip	Cassandra, Riak	No external dependency; more node-side complexity

Node IPs change slowly enough that DNS suffices for the network layer. The partition map itself lives in coordination (ZooKeeper-like) or gossip — and everyone must agree (Ch 9 consensus territory for strong guarantees).

9. Mental Models

1. Each partition is a small DB

Independence is what makes scaling linear. Anything crossing partitions (global indexes, multi-key writes) is where the pain lives.

2. Encyclopedia shelf

Key-range = sorted volumes — easy to find, but uneven unless boundaries follow the data.

3. Move the mapping, not the data

Fixed partitions and dynamic splits change partition→node only. Hash mod N entangles both mappings — catastrophic on resize.

4. Pick your pain on indexes

Local index: write-cheap, read-expensive.
Global index: read-cheap, write-expensive (usually async). No free lunch.

10. Anti-Patterns

Anti-pattern	Why it fails
hash mod N for node assignment	Nearly all keys move when cluster resizes — rebalancing catastrophe
Monotonic keys under key-range partitioning	Timestamps, auto-increment → all writes hit newest partition (hot spot)
Trusting "consistent hashing" in DB docs	Karger scheme doesn't rebalance well for databases; usually means hash partitioning with fixed partitions
Fully automatic rebalancing + auto failure detection	Slow node declared dead → load dumped on rest → cascading failure. Keep a human in the loop.
DIY secondary indexes in application code	Race conditions and partial write failures silently desync index and data

11. Worked Example: Sensor Network Compound Key

Sensor network storing measurements. Key = measurement timestamp (`year-month-day-hour-minute-second`).

Attempt 1: Key-range partitioning

Keys sorted → "all readings for March" is one cheap range scan. But writes arrive in real time — every insert lands in *today's* partition. One node saturates on writes; the rest idle. Classic hot spot from a monotonically increasing key.

Attempt 2: Hash partitioning

Hash the timestamp → writes spread uniformly. But sort order is gone: "all readings from 17:08:10 to 17:09:00" must query **every partition** (scatter/gather).

Attempt 3: Compound-key fix (Cassandra-style)

```
Primary key = (sensor_name, timestamp)
               ↑ hash → partition      ↑ sort within partition
```

With many concurrent sensors, write load spreads evenly. Per-sensor time-range queries are single-partition and efficient. Cost: multi-sensor time-range query = one range query per sensor name — you've traded a global scan for N targeted scans (fine when N is bounded).

Compound key: hash partition col + sort within

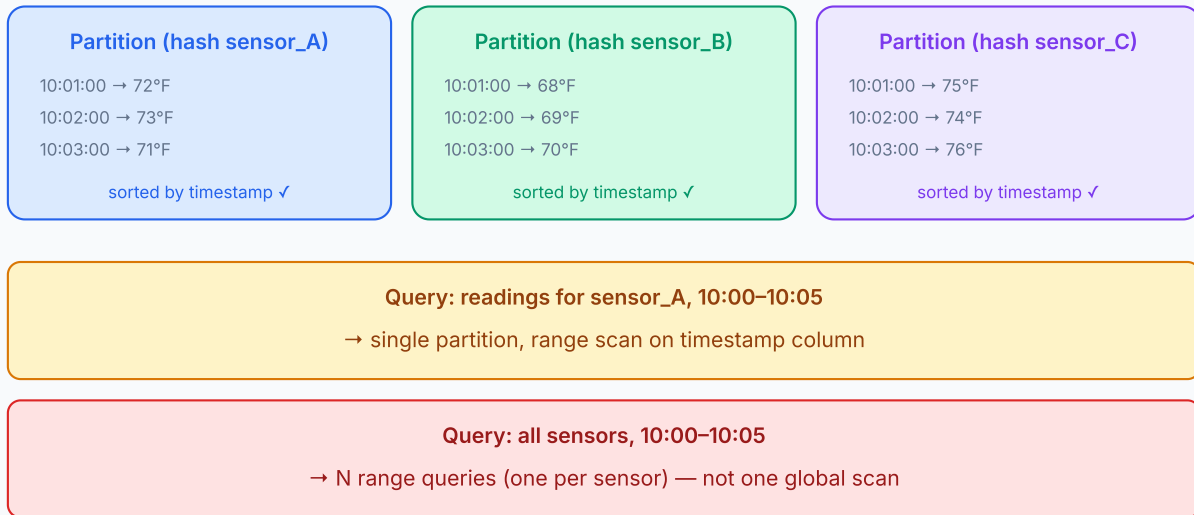


Figure 11.1 — Compound key: even write spread + efficient per-entity range scans.

Interview move: identify the hot key structure, split "which partition" from "sort within partition." Don't accept the false binary of range-vs-hash — compound keys buy both, scoped.

12. Problems Each Mechanism Solves

Mechanism	Problem it solves	Example
Consistent-hash ring + virtual nodes	Rebalance without moving most keys when node count changes	Consistent-hashing service itself; leaderless KV store's replica placement
Hash sharding + read replicas	Point-lookup index too big for one node, read-heavy	URL shortener (<code>shortURL</code> → <code>longURL</code> map)
Per-host queue partitioning	Politeness + even crawl load without a shared lock	Web crawler's URL frontier (one worker owns a host)
Compound key: hash partition col + sort/cluster col	Even write spread <i>and</i> cheap per-entity range scans	Chat (<code>channel_id</code> , <code>message_id</code>); email <code>user_id</code> partition + denormalized read/unread tables
Range partitioning + hot-range mitigation, shard-map manager	Prefix/range queries at scale without one range absorbing all load	Search autocomplete (trie shards <code>a-m</code> / <code>n-z</code> , further split on skew)
Space-filling curve (geohash / quadtree / S2)	Multi-dimensional (lat/lon) queries on a 1-D sorted-key system	Proximity service, Google Maps tile addressing
Topic partitions + consumer-group rebalancing	Parallelize an ordered log while keeping per-key order	Distributed message queue
Range-vs-hash partitioning by score, scatter/gather top-N	Global ranking query across shards without one shard owning it all	Real-time leaderboard
Consistent-hash placement groups + metadata sharded by <code>hash(bucket, object)</code>	Spread object storage across nodes while keeping listing/pagination sane	S3-like object storage

Hash sharding by natural key (<code>hotel_id</code>)	Even load without breaking single-shard-per-entity invariant transactions rely on	Hotel reservation system
Collector placement on hash ring	Avoid two collectors scraping the same target	Metrics monitoring & alerting

13. Connects To (Cross-Chapter)

Chapter	Connection	Interview soundbite
Ch 5 Replication	Partitions are replicated; each node leads some, follows others. Orthogonal choices.	"Each shard has its own leader + followers."
Ch 3 Storage	Sorted keys within a partition = SSTables; dynamic splitting mirrors B-tree node splits; concatenated/multi-column indexes.	"Partition internals look like a mini LSM/B-tree."
Ch 7 & Ch 9 Transactions / Consensus	Synchronous global index updates and agreeing on partition assignment both need distributed transactions/consensus — why global indexes go async and routing goes through ZooKeeper.	"Cross-partition = distributed transaction territory."
Ch 10 Batch	MPP warehouses parallelize joins/aggregations across partitions, far beyond single-key NoSQL access.	"Partitioning enables MPP query parallelism."

14. Key Takeaways

1. **Key-range gives cheap range scans but risks hot spots;** hash gives even load but kills range queries. Compound keys buy both, scoped: hash the first component, sort by the rest.
2. **Hashing can't fix a single hot key** — that's application-level (key salting), with read-side gather cost.
3. **Local secondary indexes:** fast writes, scatter/gather reads. **Global (term-partitioned):** fast reads, multi-partition (usually async) writes.
4. **Rebalance by moving whole partitions:** fixed-count for hash partitioning, dynamic split/merge for key ranges. **Never hash mod N.**
5. **Routing is service discovery;** the authoritative partition map lives in ZooKeeper-like coordination or gossip, and everyone must agree.
6. **Keep a human in the loop for rebalancing** — automation plus failure detection can cascade.

15. Junior SWE Checkpoints

Q1: What's the difference between replication and partitioning?

Replication **copies** the same data to multiple nodes (availability, read scaling). Partitioning **splits** data so each record lives on exactly one partition (data volume + write/query scaling). Usually combined: each node leads some partitions, follows others.

Q2: Why is timestamp-as-primary-key bad for key-range partitioning?

Timestamps are monotonically increasing — all new writes land on the newest partition (today's range). One node saturates; rest idle. Classic hot spot. Fix: compound key (hash entity ID, sort by timestamp) or hash the timestamp if range scans aren't needed.

Q3: Hash partitioning spreads load but breaks what?

Range queries on the primary key. Adjacent keys scatter to different partitions — "user:40 to user:50" must query all partitions (scatter/gather). Hash also can't fix a single hot key (celebrity ID) — same hash every time.

Q4: Explain compound key (user_id, timestamp) in one sentence.

Hash `user_id` to pick the partition (even spread across users); sort by `timestamp` within the partition (cheap per-user time-range scans).

Q5: Local vs global secondary index — what's the trade?

Local (document-partitioned): write touches one partition; read scatter/gathers all partitions. **Global (term-partitioned):** read hits one partition; write may fan out to many index partitions → typically async (DynamoDB GSIs).

Q6: Why is $\text{hash}(\text{key}) \bmod N$ an anti-pattern?

When N (node count) changes, nearly every key's partition assignment changes — catastrophic rebalancing. Fixed partition count (move partition→node only) or dynamic split/merge avoids this.

Q7: Sensor network — walk through the three partitioning attempts.

(1) Key-range on timestamp: cheap range scans, write hot spot on today's partition. (2) Hash timestamp: even writes, no cheap range scans. (3) Compound $(\text{sensor_name}, \text{timestamp})$: hash sensor for even writes, sort by timestamp for per-sensor range queries. Multi-sensor global query = N targeted scans.

Glossary

Term	Definition
Partition / shard	Subset of data; each record belongs to exactly one
Skew	Uneven load distribution across partitions
Hot spot	Partition with disproportionately high load
Key-range partitioning	Contiguous sorted key ranges per partition — cheap range scans, hot-spot risk
Hash partitioning	Hash key to pick partition — even load, no primary-key range queries
Compound primary key	Hash first column → partition; sort remaining columns within
Key salting	Append random suffix to hot key to spread writes (read gather cost)
Local secondary index	Index per partition — fast writes, scatter/gather reads
Global secondary index	Index partitioned by term — fast reads, multi-partition async writes
Scatter/gather	Query all partitions in parallel, merge results client-side
Fixed partitions	Many more partitions than nodes; rebalance moves partition→node only
Dynamic partitioning	Split/merge partitions by size threshold (B-tree-like)
hash mod N	Anti-pattern — most keys move when N changes
Pre-splitting	Create initial partition boundaries before data arrives (avoid single hot partition)